Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Unit 3 - Linked Lists

Need for Linked Lists

For storing similar types of data in memory, we can use either an array or a linked list. Arrays are simple to understand, and accessing their elements is fast and straightforward. However, they have some important limitations:

- **Fixed size:** Once the size of an array is decided, it cannot be changed during program execution.
- **Costly insertion:** Inserting a new element at a specific position requires shifting all subsequent elements one step to the right.
- **Costly deletion:** Deleting an element requires shifting all subsequent elements one step to the left.

Linked lists overcome these limitations. A linked list can grow or shrink dynamically during its lifetime, so there is no fixed maximum size and no memory wastage. Also, unlike arrays, inserting or deleting elements does not require shifting other elements, making these operations more efficient in many cases.

A Linked List is a linear data structure where elements (nodes) are stored at non-contiguous memory locations. A linked list is a collection of nodes connected through pointers (links).
Each node has two main parts:

1. **Data Field** – stores the actual data (value) of the node.
2. **Pointer (Link) Field** – stores the address (reference) of the next node in the list.

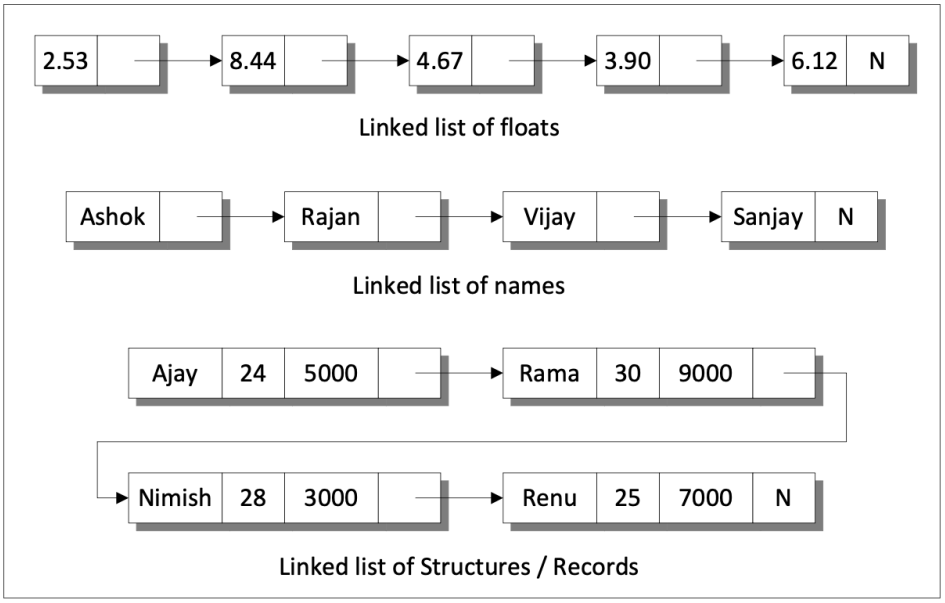
Types of Linked Lists

Based on the way nodes are connected, linked lists can be classified as follows:

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List
    a. Singly Circular Linked List
    b. Doubly Circular Linked List

More Linked Lists

A common misconception among beginners is that a linked list can only store integers.
In reality, a linked list can store any type of similar data. For example, you can have a linked list of floating-point numbers, a linked list of names, or even a linked list of complex records—such as employee details containing name, age, and salary.



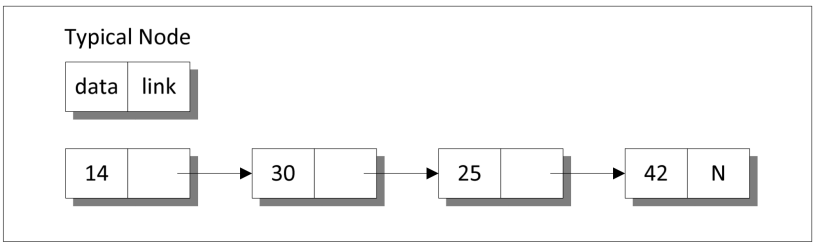
Singly Linked List

A singly linked list is a fundamental data structure, it consists of nodes where each node contains a data field and a reference to the next node in the linked list.

- ➔ The first node is called the ***Head*** (starting point of the list).
- ➔ The last node's pointer is set to ***NULL*** (indicating the end of the list).
- ➔ Traversal is possible only in one direction (forward).

Syntax

```
struct Node {
    int data;              // Data part
    struct Node* next;     // Pointer to the next node
};
```



Advantages: Simple to implement, efficient insertion/deletion at the beginning.
Disadvantages: Cannot traverse backward.
Use Case: Stacks, simple dynamic lists.

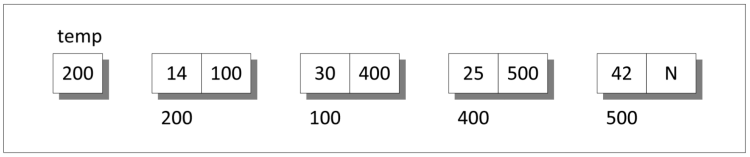


Figure 3-3. Connection of nodes.

```
// Function to create a new Node
struct Node* newNode(int data) {
    struct Node* temp =
      (struct Node*)malloc(sizeof(struct Node));
    temp->data = data;
    temp->next = NULL;
    return temp;
}
```

Operations on a Singly Linked List

A singly linked list consists of nodes, where each node contains data and a pointer to the next node. The head pointer stores the address of the first node in the list.

Traversal of Singly Linked List

Traversal in a singly linked list means visiting each node of the list one by one in order to perform certain operations such as displaying elements, searching for a value, or counting the number of nodes.
Since a singly linked list can only be traversed in one direction (from head node to the last node), we always start from the head pointer and follow the chain of links until we reach NULL.

Algorithm: Traverse_Singly_Linked_List(HEAD)

1. Start.
2. Set ptr = HEAD.
3. Repeat steps 4 and 5 while ptr != NULL.
4. Process (Display) ptr → data.
5. Set ptr = ptr → next.
6. Stop.

Code

```
// Function to traverse and print linked list
void traverse(struct Node* head) {
    struct Node* temp = head;   // Start from head
    while (temp != NULL) {
        printf("%d -> ", temp->data);  // Process data
        temp = temp->next;             // Move to next node
    }
    printf("NULL\n"); // End of list
}
```

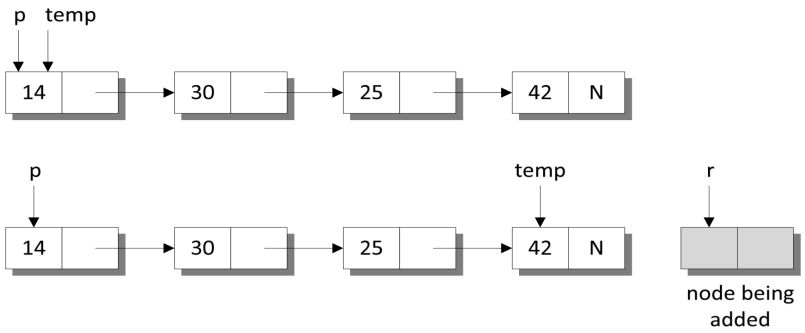
Traversal of a singly linked list is used in many operations such as:

- Displaying the elements of the list.
- Searching for a given element.
- Counting the number of nodes in the list.
- Performing arithmetic operations (sum, average, maximum, minimum, etc.) on the list elements.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Append (Add a node at the end)

Appending means adding a new node at the end of the linked list. Create a new node and allocate memory dynamically. Store the given data in the new node. If the list is empty, the new node itself becomes the head node. If the list is not empty, we must traverse the entire list until we reach the last node (where next = NULL) and then link the new node there.



Algorithm: Append(HEAD, ITEM)

1. Start.
2. Create a new node NEW.
3. Set NEW → data = ITEM.
4. Set NEW → next = NULL.
5. If HEAD == NULL, then
    a. Set HEAD = NEW.
    b. Stop.
6. Else
    a. Set PTR = HEAD.
    b. Repeat while PTR → next != NULL
        i. Set PTR = PTR → next.
    c. Set PTR → next = NEW.
7. Stop.

Code

```
// Function to append node at the end
void append(struct Node** head_ref, int new_data) {
    // Step 1: Create new node
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = NULL;

    // Step 2: If list is empty, new node becomes head
    if (*head_ref == NULL) {
        *head_ref = new_node;
        return;
    }
```

```
    // Step 3: Traverse till the last node
    struct Node* last = *head_ref;
    while (last->next != NULL) {
        last = last->next;
    }

    // Step 4: Make last node point to new node
    last->next = new_node;
}
```

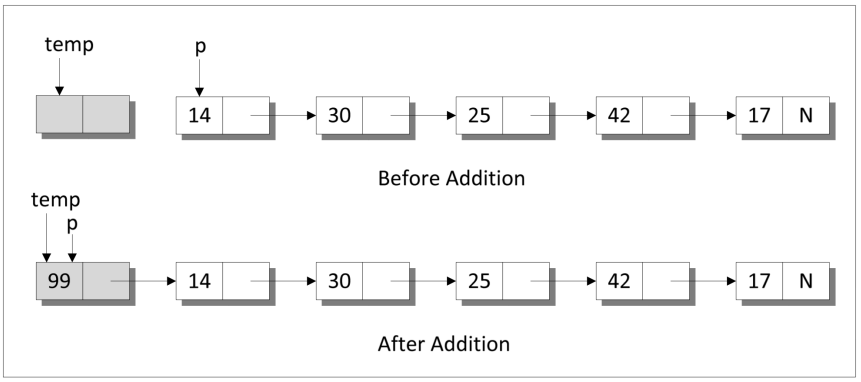
Appending is useful when:

- We want to insert elements in sequence as they arrive.
- Implementing queues using linked lists (enqueue operation).
- Creating dynamic lists where new data is always added at the end.

Add at the Beginning(Prepend Operation)

Adding a node at the beginning of a singly linked list is called insertion at the beginning or prepending. The process of adding a node at the beginning of a singly linked list starts with creating a new node and allocating memory for it dynamically. The given data is then stored in the data part of this new node. After that, the next pointer of the new node is set to point to the current head node of the list. Finally, the head pointer is updated so that it points to the new node. After completing these steps, the new node becomes the first node of the linked list. This operation is very fast because we do not need to traverse the entire list; we only update pointers.

Insertion at the beginning is useful when we want to create a list in reverse order of input, when implementing a stack using a linked list (push operation), or when frequent insertions at the start of the list are required.



Algorithm: Insert_Begin(HEAD, ITEM)

1. Start.
2. Create a new node NEW.
3. Set NEW → data = ITEM.
4. Set NEW → next = HEAD.
5. Set HEAD = NEW.
6. Stop.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

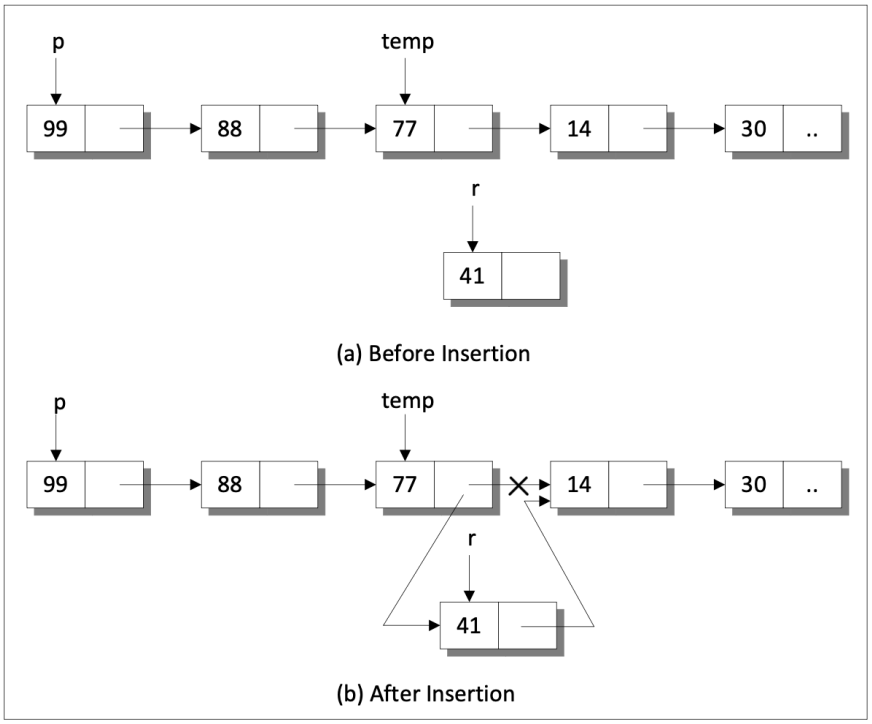
Code

```
// Function to insert node at beginning
void insertAtBeginning(struct Node** head_ref, int new_data) {
    // Step 1: Create new node
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;

    // Step 2: Link new node to current head
    new_node->next = *head_ref;

    // Step 3: Move head to point to new node
    *head_ref = new_node;
}
```

Add After N Nodes

Sometimes we may need to insert a new node after a specific position in a linked list. For example, if the list is 10 → 20 → 30 → NULL and we insert 15 after the first node, the new list becomes 10 → 15 → 20 → 30 → NULL. This operation is useful for adding nodes in the middle of the list rather than only at the beginning or the end. To perform this, we create a new node, store the data in it, and traverse the list until the required position is reached. Then, the new node's next pointer is set to the node that originally followed, and the previous node's next pointer is updated to link to the new node. In this way, the new node is successfully inserted after the specified position.

It is useful when we want to insert data at a specific position in the list, when it is necessary to maintain sorted or ordered data during insertion, or when implementing real-time dynamic structures such as priority-based queues and linked memory allocations.



Algorithm: Insert_After_N(HEAD, N, ITEM)

1. Start.
2. Create a new node NEW.
3. Set NEW → data = ITEM.
4. Set PTR = HEAD.
5. Repeat N–1 times: PTR = PTR → next (to reach the Nth node).
6. Set NEW → next = PTR → next.
7. Set PTR → next = NEW.
8. Stop.

Code

```
// Function to insert after N nodes
void insertAfterN(struct Node* head, int position, int new_data) {
    // Step 1: Create new node
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;

    // Step 2: Traverse to Nth node
    struct Node* temp = head;
    for (int i = 1; i < position && temp != NULL; i++) {
        temp = temp->next;
    }

    // If position is invalid
    if (temp == NULL) {
        printf("The given position does not exist in the list.\n");
        free(new_node);
        return;
    }

    // Step 3: Insert new node
    new_node->next = temp->next;
    temp->next = new_node;
}
```

Search an Element

Given a singly linked list and an element (key), the task is to check whether the element exists in the list. If it is found, return its position; otherwise, return "not found."

Algorithm: Search(HEAD, ITEM)

1. Start.
2. Set PTR = HEAD and POS = 1.
3. Repeat steps 4–6 while PTR != NULL.
4. If PTR → data == ITEM, then

   a. Print "Element found at position POS".
   b. Stop.
5. Else
   a. Set PTR = PTR → next.
   b. Increment POS = POS + 1.
6. Continue until the end of the list.
7. If loop ends and element not found, print "Element not found in the list".
8. Stop.

Finding the Length

The length of a linked list means the total number of nodes present in the list. To count the nodes, we need to traverse the list from the head node to the last node and maintain a counter that increases with each node visited.

Algorithm: Length(HEAD)

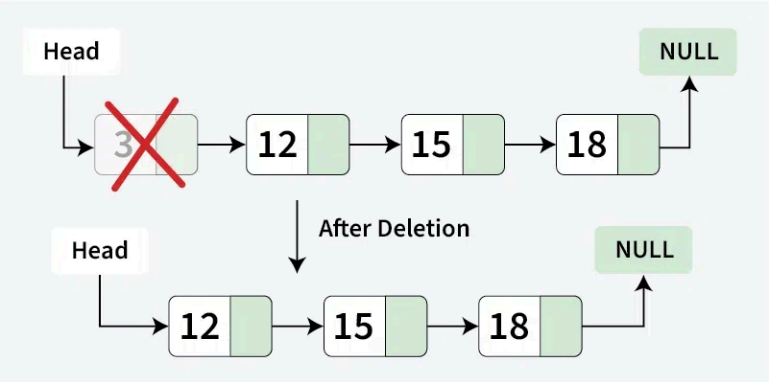
1. Start.
2. Set PTR = HEAD and COUNT = 0.
3. Repeat steps 4 and 5 while PTR != NULL.
4. Increment COUNT = COUNT + 1.
5. Move to the next node: PTR = PTR → next.
6. When traversal ends, print COUNT as the total number of nodes.
7. Stop.

Deletion from the Beginning

Deletion from the beginning of a singly linked list means removing the first node, also called the head node. After deletion, the second node becomes the new head, and if the list is empty, no deletion is possible. To perform this operation, we first check if the list is empty; if it is, we display a message. Otherwise, we make a temporary pointer point to the head node, move the head pointer to the next node, and free the memory of the old head node. After these steps, the list successfully starts from the new head node.

Algorithm: Delete_Begin(HEAD)

1. Start.
2. If HEAD == NULL, then print "List is empty, deletion not possible" and stop.
3. Set TEMP = HEAD.
4. Set HEAD = HEAD → next.
5. Free the node pointed by TEMP.
6. Stop.



Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

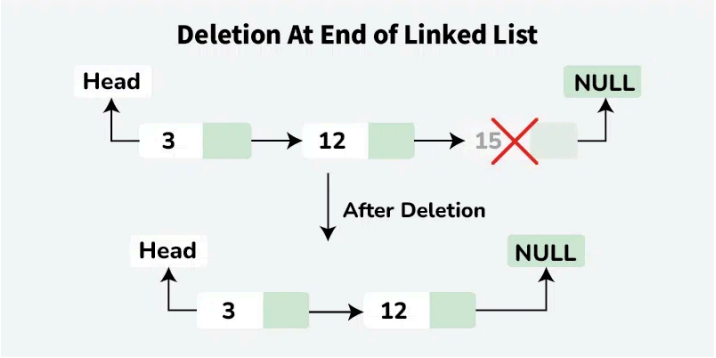
Code

```
// Function to delete node from beginning
void deleteBeginning(struct Node** head_ref) {
    if (*head_ref == NULL) {
        printf("List is empty, deletion not possible.\n");
        return;
    }

    // Step 1: Store head node in temp
    struct Node* temp = *head_ref;

    // Step 2: Move head to next node
    *head_ref = (*head_ref)->next;

    // Step 3: Free memory of old head
    free(temp);
}
```

Deletion from the beginning is commonly used in stack implementation for the **pop operation**, where elements are removed one by one. It is also useful in scheduling or buffer applications where the first arrived data needs to be removed to maintain proper order of processing.

Deletion at the End

Deletion at the end of a singly linked list means removing the last node. To perform this, we first check if the list is empty (no deletion possible). If there is only one node, deleting it makes the list empty. Otherwise, we traverse the list until the second last node, make its next pointer NULL, and free the last node. This operation is useful when we need to remove the most recently added data in certain dynamic lists or in queue variations where deletion occurs from the rear.



Algorithm: Delete_End(HEAD)

1. Start.
2. If HEAD == NULL, print "List is empty, deletion not possible" and stop.
3. If HEAD → next == NULL, free HEAD and set HEAD = NULL, then stop.
4. Set PTR = HEAD.

5. Repeat while PTR → next → next != NULL
    a. Set PTR = PTR → next.
6. Free the node PTR → next.
7. Set PTR → next = NULL.
8. Stop.

Code

```
// Function to delete node from end
void deleteEnd(struct Node** head_ref) {
    if (*head_ref == NULL) {
        printf("List is empty, deletion not possible.\n");
        return;
    }

    // If only one node in the list
    if ((*head_ref)->next == NULL) {
        free(*head_ref);
        *head_ref = NULL;
        return;
    }

    // Traverse to second last node
    struct Node* temp = *head_ref;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    // Free last node
    free(temp->next);
    temp->next = NULL;
}
```

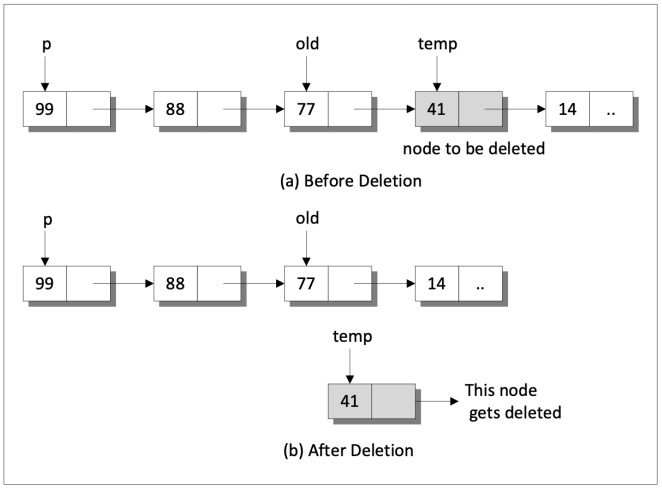
Delete a Specific Node (by Value)

Deleting a specific node by value means finding the node that contains the given data and then removing it from the linked list. The steps are: first, check if the list is empty. If the head node itself contains the value, update the head pointer to the next node and free the old head. Otherwise, traverse the list using a pointer to find the node just before the one to be deleted. Then, adjust the next pointer to skip the target node, and finally free the memory of that node. This operation is useful when we want to remove a particular element from a dataset, such as deleting a customer record, canceling a specific job in scheduling, or removing unwanted values from dynamic data structures.

Algorithm: Delete_By_Value(HEAD, VALUE)

1. Start.
2. If HEAD == NULL, print "List is empty, deletion not possible" and stop.

3. If HEAD → data == VALUE:
    a. Set TEMP = HEAD.
    b. Set HEAD = HEAD → next.
    c. Free TEMP.
    d. Stop.
4. Set PTR = HEAD.
5. Repeat while PTR → next != NULL and PTR → next → data != VALUE:
    a. Set PTR = PTR → next.
6. If PTR → next == NULL, print "Value not found" and stop.
7. Set TEMP = PTR → next.
8. Set PTR → next = TEMP → next.
9. Free TEMP.
10. Stop.



Code

```
// Function to delete a node by value
void deleteByValue(struct Node** head_ref, int value) {
    if (*head_ref == NULL) {
        printf("List is empty, deletion not possible.\n");
        return;
    }

    struct Node* temp = *head_ref;

    // If head node holds the value
    if (temp->data == value) {
        *head_ref = temp->next;
        free(temp);
        return;
    }
    // Search for the node to be deleted
    struct Node* prev = NULL;
```

```
    while (temp != NULL && temp->data != value) {
        prev = temp;
        temp = temp->next;
    }
    // Value not found
    if (temp == NULL) {
        printf("Value %d not found in the list.\n", value);
        return;
    }
    // Unlink the node
    prev->next = temp->next;
    free(temp);
}
```

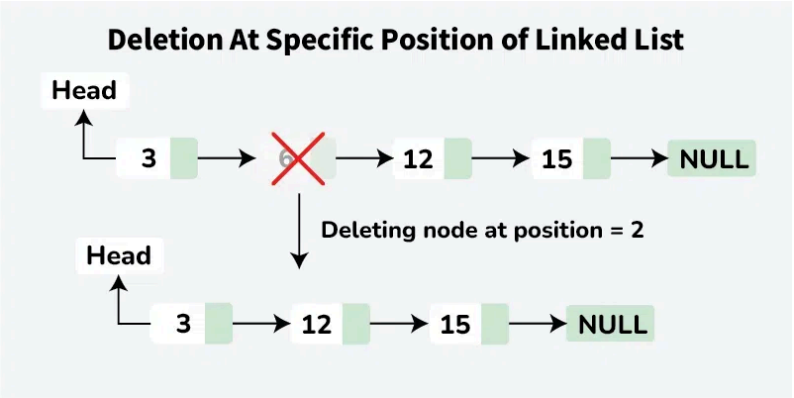
Deletion at a Specific Position

Deletion at a specific position means removing the node present at a given index or position in the linked list. The steps are: first, check if the list is empty. If the position is 1, then delete the head node by updating the head pointer to the next node and freeing the old head. For other positions, traverse the list until you reach the node just before the target position, adjust its next pointer to skip the node to be deleted, and free the memory of that node. This operation is useful when we want to manage dynamic data structures where deletion is required at arbitrary positions, such as updating records in databases, memory management, or task scheduling systems.

Algorithm: Delete_Position(HEAD, POS)

1. Start.
2. If HEAD == NULL, print "List is empty, deletion not possible" and stop.
3. If POS == 1:
    a. Set TEMP = HEAD.
    b. Set HEAD = HEAD → next.
    c. Free TEMP.
    d. Stop.
4. Set PTR = HEAD.
5. Repeat for i = 1 to POS - 2:
    a. If PTR == NULL or PTR → next == NULL, print "Position out of range" and stop.
    b. Else set PTR = PTR → next.
6. Set TEMP = PTR → next.
7. If TEMP == NULL, print "Position out of range" and stop.
8. Set PTR → next = TEMP → next.
9. Free TEMP.
10. Stop.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)



Code

```
// Function to delete node at a specific position
void deleteAtPosition(struct Node** head_ref, int position) {
    if (*head_ref == NULL) {
        printf("List is empty, deletion not possible.\n");
        return;
    }

    struct Node* temp = *head_ref;

    // If head needs to be removed
    if (position == 1) {
        *head_ref = temp->next;
        free(temp);
        return;
    }

    // Traverse to the node before the target position
    struct Node* prev = NULL;
    for (int i = 1; temp != NULL && i < position; i++) {
        prev = temp;
        temp = temp->next;
    }

    // If position is out of range
    if (temp == NULL) {
        printf("Position %d is out of range.\n", position);
        return;
    }

    // Unlink and free node
    prev->next = temp->next;
    free(temp);
}
```

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Reverse a Singly Linked List

Reversing a singly linked list means changing the direction of all its links so that the head node becomes the last node, and the last node becomes the head. The steps are: initialize three pointers — prev, current, and next. Start with current as the head and prev as NULL. For each node, store the next node in next, change the current → next pointer to point to prev, then move prev and current one step forward. After the loop ends, update the head pointer to prev. This operation is useful in many real-world applications like undo functionality in editors, backtracking problems, and data manipulation tasks where the order of elements needs to be reversed.

Algorithm: Reverse_List(HEAD)

1. Start.
2. Initialize prev = NULL, current = HEAD, next = NULL.
3. While current != NULL:
    a. Set next = current → next.
    b. Set current → next = prev.
    c. Set prev = current.
    d. Set current = next.
4. Set HEAD = prev.
5. Stop.

Concatenate Two Singly Linked Lists

Concatenation of two singly linked lists means joining the second list to the end of the first list, creating a single continuous list. The process involves checking if the first list is empty; if it is, the head of the first list becomes the head of the second list. Otherwise, we traverse the first list until we reach its last node, then update its next pointer to point to the head of the second list. This operation is useful in scenarios where separate lists of data need to be merged, such as merging student records, combining tasks, or joining datasets in memory.

Algorithm: Concatenate_List(HEAD1, HEAD2)

1. Start.
2. If HEAD1 == NULL, then set HEAD1 = HEAD2 and stop.
3. Else, set PTR = HEAD1.
4. While PTR → next != NULL:
    a. Set PTR = PTR → next.
5. Set PTR → next = HEAD2.
6. Stop.

Singly Linked List (Array Representation)

In array representation of a singly linked list, two parallel arrays are maintained: DATA[] to store node values and NEXT[] to store indices of the next node. The START (or HEAD) variable holds the index of the first node, and AVAIL points to the beginning of the free list (unused positions). Each node is represented by an array index instead of a pointer. Traversal

continues using NEXT[] values until -1 is encountered, which marks the end of the list. This representation is useful in environments where pointers are not allowed (such as some high-level programming languages or restricted memory systems), and it helps in simulating linked list operations like traversal, insertion, and deletion using arrays.

Index	0	1	2	3	4
DATA	10	25	15	—	20
NEXT	2	-1	4	—	1

- START/HEAD = 0 → The list starts at index 0.
- AVAIL = 3 → Index 3 is the first free node.
- Traversal (following NEXT) → 10 → 15 → 20 → 25

Algorithm: Traverse_Array_List(DATA[], NEXT[], HEAD)

1. Start.
2. Set PTR = HEAD.
3. While PTR != -1:
    a. Print DATA[PTR].
    b. Set PTR = NEXT[PTR].
4. Stop.

Insertion Operations

(a) Insert at Beginning

A new node is inserted before the current START. Its NEXT points to the old START, and then START is updated to this new node.

Algorithm (Insert_Begin)

1. If AVAIL == -1, report Overflow.
2. NEW = AVAIL, AVAIL = NEXT[AVAIL].
3. DATA[NEW] = ITEM.
4. NEXT[NEW] = START.
5. START = NEW

Example (Insert 5 at Beginning)

Index	0	1	2	3	4
DATA	10	25	15	5	20
NEXT	2	-1	4	0	1

Traversal: 5 → 10 → 15 → 20 → 25

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

(b) Insert at End

Traverse to the last node (NEXT = -1), then attach a new node from AVAIL at the end.

Algorithm (Insert_End)

1. If AVAIL == -1, report Overflow.
2. NEW = AVAIL, AVAIL = NEXT[AVAIL].
3. DATA[NEW] = ITEM, NEXT[NEW] = -1.
4. Traverse to last node.
5. NEXT[last] = NEW.

Example (Insert 5 at End)

Index	0	1	2	3	4
DATA	10	25	15	5	20
NEXT	2	3	4	-1	1

Traversal: 10 → 15 → 20 → 25→ 5

(c) Insert After a Node

Find the target node, create a new node from AVAIL, link it after the target by adjusting NEXT.

Algorithm (Insert_After)

1. If AVAIL == -1, report Overflow.
2. NEW = AVAIL, AVAIL = NEXT[AVAIL].
3. Traverse to target node LOC.
4. DATA[NEW] = ITEM.
5. NEXT[NEW] = NEXT[LOC].
6. NEXT[LOC] = NEW.

Example (Insert 30 after 15)

Index	0	1	2	3	4
DATA	10	25	15	30	20
NEXT	2	-1	3	4	1

Traversal: 10 → 15 → 30 → 20→2 5

Deletion Operations

(a) Delete from Beginning

The START node is removed; the next node becomes the new START.

Algorithm (Delete_Begin)

1. If START == -1, report Underflow.
2. PTR = START.
3. START = NEXT[START].
4. Release PTR to AVAIL

(b) Delete from End

Traverse to the second-last node and set its NEXT = -1, releasing the last node.

Algorithm (Delete_End)

1. If START == -1, report Underflow.
2. Traverse to second-last node.
3. PTR = NEXT[second_last].
4. NEXT[second_last] = -1.
5. Release PTR to AVAIL.

(c) Delete a Specific Node

Find the node before the target, bypass the target node by updating its NEXT, then release the target to AVAIL.

Algorithm (Delete_Node)

1. If START == -1, report Underflow.
2. Traverse until NEXT[ptr] = target.
3. NEXT[ptr] = NEXT[target].
4. Release target to AVAIL.

Advantages

The array representation of linked lists avoids dynamic memory allocation, which is useful in systems where malloc or free is not available. Memory management is easier since all nodes are stored within a fixed-size array, and array indexing provides faster access compared to pointer dereferencing.

Limitations

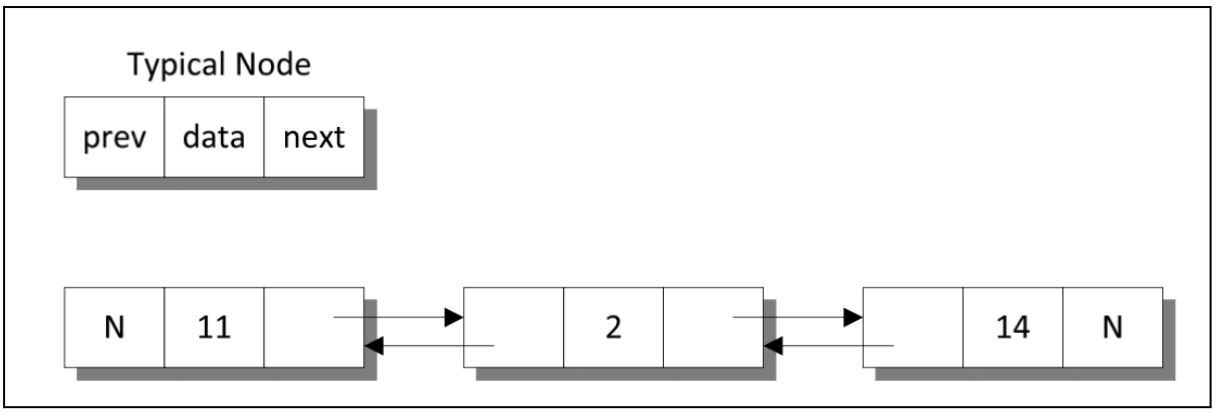
The major limitation of array representation is fixed size — the maximum number of nodes must be predefined, making it less flexible than pointer-based linked lists.

Doubly Linked List (DLL)

In the linked lists we have used so far, each node only stores information about the next node in the sequence. It has no knowledge of where the previous node is located in memory.
This means that if we are at, say, the 15th node and want to reach the 14th node, we must traverse the list all the way from the first node.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

To overcome this limitation, we can modify each node to store two addresses — one for the next node and another for the previous node. Such an arrangement is called a Doubly Linked List.



Structure of a Node (DLL)

```
struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};
```

A Doubly Linked List (DLL) is a type of linked list in which each node contains three parts:

- **DATA field** → stores the actual information.
- **PREV pointer** → stores the address (or index) of the previous node.
- **NEXT pointer** → stores the address (or index) of the next node.

Thus, every node in a DLL is connected in both directions, allowing traversal from left to right (***forward direction***) as well as from right to left (***backward direction***).

- The first node of the list has PREV = NULL and the last node has NEXT = NULL.
- A TAIL pointer is a special pointer that always points to the last node of the linked list.

Advantages of DLL

- Traversal can be done in both directions (forward and backward).
- Deletion is more efficient because we can directly access the previous node using PREV.
- Insertion and deletion at both ends (beginning and end) are faster than in singly linked lists, if we maintain a TAIL pointer, we can directly insert/delete from end without traversal.
- It is useful for applications requiring bidirectional navigation, such as undo/redo functionality.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Limitations of DLL

- Requires extra memory for storing the PREV pointer in every node.
- Insertion and deletion involve more pointer adjustments compared to singly linked lists.
- Implementation is more complex due to handling two pointers per node.

Applications of DLL

- Navigation systems (e.g., forward and backward movement in web browsers, image viewers, or playlists).
- Undo/Redo functionality in text editors and word processors.
- Deque (double-ended queue) implementation.
- Polynomial representation and manipulation.

Algorithms for Doubly Linked List

Traversal (Forward)

Algorithm: TraverseDLL(head)

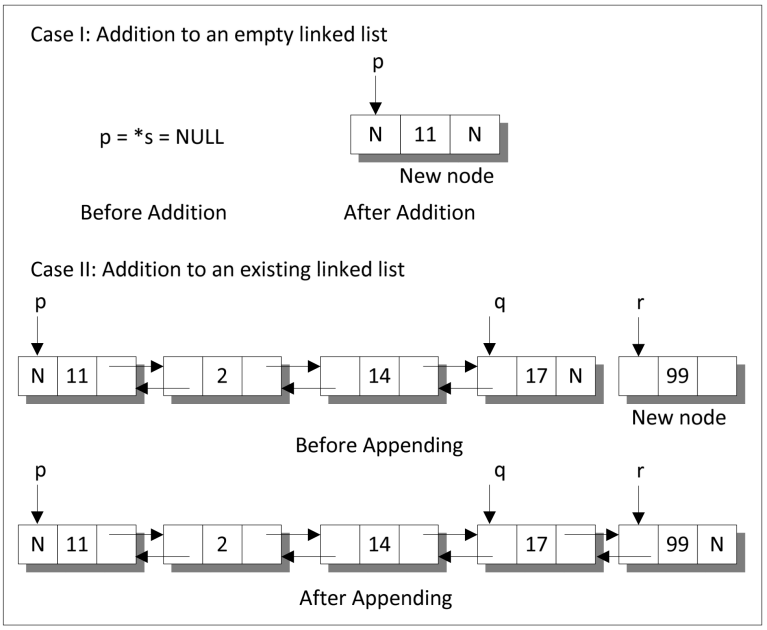
1. Set PTR = head.
2. Repeat while PTR != NULL:
3. Print PTR->DATA.
4. Move PTR = PTR->NEXT.
5. Exit.

Traversal (Backward)

Algorithm: Traverse_Backward(HEAD)

1. IF HEAD = NULL
    a. Print "List is Empty" and EXIT.
2. Set PTR = HEAD.
3. WHILE PTR->NEXT ≠ NULL
    a. PTR = PTR->NEXT

    [Now PTR points to the last node]
4. WHILE PTR ≠ NULL
    a. PROCESS PTR->DATA [e.g.,
    b. PTR = PTR->PREV
5. STOP.



Insertion at Beginning

Algorithm: InsertBegin(head, ITEM)

1. Create a new node NEW.
2. Set NEW->DATA = ITEM.
3. Set NEW->PREV = NULL.
4. Set NEW->NEXT = head.

5. If head != NULL
    - head->PREV = NEW.
6. Update head = NEW.
7. Exit.

Insertion at End

Algorithm: InsertEnd(head, ITEM)

1. Create a new node NEW.
2. Set NEW->DATA = ITEM.
3. Set NEW->NEXT = NULL.
4. If head == NULL:
    a. Set NEW->PREV = NULL.
    b. Set head = NEW.
    c. Exit.
5. Else:
    a. Set PTR = head
    b. Repeat while PTR->NEXT != NULL:
        - PTR = PTR->NEXT.
    c. Set PTR->NEXT = NEW.
    d. Set NEW->PREV = PTR.
6. Exit.

Insertion After a Specific Node

Algorithm: InsertAfter(head, ITEM, KEY)

1. Set PTR = head.
2. Traverse until PTR != NULL and PTR->DATA != KEY:
    - PTR = PTR->NEXT.
3. If PTR == NULL, print "Node not found" and exit.
4. Create a new node NEW.
5. Set NEW->DATA = ITEM.
6. Set NEW->NEXT = PTR->NEXT.
7. Set NEW->PREV = PTR.
8. If PTR->NEXT != NULL, then set PTR->NEXT->PREV = NEW.
9. Set PTR->NEXT = NEW.
10. Exit.

Deletion from Beginning

Algorithm: DeleteBegin(head)

1. If head == NULL, print "List Empty" and exit.
2. Set PTR = head.
3. Set head = head->NEXT.
4. If head != NULL, set head->PREV = NULL.
5. Free PTR.

6. Exit.

Deletion from End

Algorithm: DeleteEnd(head)

1. If head == NULL, print "List Empty" and exit.
2. Set PTR = head.
3. Repeat while PTR->NEXT != NULL:
   - PTR = PTR->NEXT.

   (Now PTR is the last node)
4. If PTR->PREV != NULL, set PTR->PREV->NEXT = NULL.
5. Else (only one node): set head = NULL.
6. Free PTR.
7. Exit.

Deletion of a Specific Node (by Value)

Algorithm: DeleteNode(head, KEY)

1. If head == NULL, print "List Empty" and exit.
2. Set PTR = head.
3. Traverse until PTR != NULL and PTR->DATA != KEY:
   - PTR = PTR->NEXT.
4. If PTR == NULL, print "Node not found" and exit.
5. If PTR->PREV != NULL, set PTR->PREV->NEXT = PTR->NEXT.
   Else set head = PTR->NEXT.
6. If PTR->NEXT != NULL, set PTR->NEXT->PREV = PTR->PREV.
7. Free PTR.
8. Exit.

Array Representation of Doubly Linked List

In a doubly linked list, each node contains three fields: the data field, the previous link (PREV) that points to the address of the previous node, and the next link (NEXT) that points to the address of the next node. Instead of using actual memory addresses and pointers (as in dynamic linked lists), we can represent a doubly linked list using parallel arrays, where each index corresponds to a node, and the PREV[] and NEXT[] arrays store the links in terms of indices rather than addresses.

In this representation, three arrays are maintained:

- **DATA[]** → stores the values of the nodes.
- **PREV[]** → stores the index of the previous node in the list.
- **NEXT[]** → stores the index of the next node in the list.

We also use two special pointers:

- **START** → stores the index of the first node in the list.
- **AVAIL** → points to the first free index in the memory pool, which can be used for future insertions.

Also, -1 → indicates null pointer

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Node Representation

Suppose we have 5 available locations, and the linked list is stored as follows:

Index	**0**	**1**	**2**	**3**	**4**
DATA	10	25	15	—	20
PREV	-1	4	0	—	2
NEXT	2	-1	4	—	1

Here,

- START = 0, meaning the list starts from index 0.
- AVAIL = 3, meaning index 3 is the first free node available for insertion.

The traversal of the list can be done by following the NEXT[] links:

10 → 15 → 20 → 25.

Also, using the PREV[] array, we can traverse backward:

25 → 20 → 15 → 10.

Forward Traversal Algorithm

1. Set PTR = START.
2. Repeat while PTR ≠ -1:
3. Process DATA[PTR].
4. Set PTR = NEXT[PTR].

Backward Traversal Algorithm

1. Traverse forward until the last node is found (NEXT[PTR] = -1).
2. Repeat while PTR ≠ -1:
3. Process DATA[PTR].
4. Set PTR = PREV[PTR].

Advantages of Array Representation of DLL

1. Memory can be pre-allocated, which makes access by index very efficient.
2. Arrays avoid pointer-related errors since links are stored as indices.
3. Traversal both forward and backward is easy using NEXT[] and PREV[].
4. Implementation is straightforward in languages without dynamic memory management.

Limitations of Array Representation of DLL

1. Memory is fixed and cannot grow dynamically like a pointer-based linked list.
2. Insertion and deletion require updating multiple arrays (DATA[], PREV[], NEXT[]), which increases overhead.
3. Managing the AVAIL list for free memory requires extra bookkeeping.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Insertion at Beginning

Algorithm Insert_Beginning(DATA[], PREV[], NEXT[], START, AVAIL, ITEM)

1. If AVAIL = -1 then
    - Print "Overflow" and Exit
2. NEW = AVAIL
3. DATA[NEW] = ITEM
4. AVAIL = NEXT[AVAIL] // Move AVAIL to next free node
5. NEXT[NEW] = START
6. PREV[NEW] = -1
7. If START != -1 then
    - PREV[START] = NEW
8. START = NEW
9. Exit

Insertion at End

Algorithm: Insert_End(DATA, NEXT, PREV, START, AVAIL, ITEM)

1. If AVAIL = -1 then
    - Print "Overflow" and Exit
2. NEW = AVAIL
3. DATA[NEW] = ITEM
4. AVAIL = NEXT[AVAIL] // Update free list
5. NEXT[NEW] = -1
6. If START = -1 then
    - START = NEW
    - PREV[NEW] = -1
    - Exit
7. PTR = START
8. Repeat while NEXT[PTR] != -1
    - PTR = NEXT[PTR]
9. NEXT[PTR] = NEW
10. PREV[NEW] = PTR
11. Exit

Algorithm: Insert_After(DATA, NEXT, PREV, START, AVAIL, POS, ITEM)

1. If AVAIL = -1 then
    - Print "Overflow" and Exit
2. PTR = START
3. COUNT = 1
4. Repeat while PTR != -1 and COUNT < POS
    - PTR = NEXT[PTR]
    - COUNT = COUNT + 1
5. If PTR = -1 then
    - Print "Invalid Position" and Exit

6. NEW = AVAIL
7. DATA[NEW] = ITEM
8. AVAIL = NEXT[AVAIL] // Move AVAIL forward
9. NEXT[NEW] = NEXT[PTR]
10. PREV[NEW] = PTR
11. If NEXT[PTR] != -1 then
    - PREV[NEXT[PTR]] = NEW
12. NEXT[PTR] = NEW
13. Exit

Deletion at Beginning

Algorithm: Delete_Begin(DATA, NEXT, PREV, START, AVAIL)

1. If START = -1 then
    - Print "Underflow" and Exit
2. PTR = START
3. START = NEXT[START]
4. If START != -1 then
    - PREV[START] = -1
5. NEXT[PTR] = AVAIL // Release node
6. AVAIL = PTR
7. Exit

Deletion at End

Algorithm: Delete_End(DATA, NEXT, PREV, START, AVAIL)

1. If START = -1 then
    - Print "Underflow" and Exit
2. PTR = START
3. Repeat while NEXT[PTR] != -1
    - PREV_PTR = PTR
    - PTR = NEXT[PTR]
4. If PTR = START then // Only one node in list
    - START = -1

   Else
    - NEXT[PREV_PTR] = -1
5. NEXT[PTR] = AVAIL // Release node
6. AVAIL = PTR
7. Exit

Deletion of a Specific Node (by value)

Algorithm: Delete_Node(DATA, NEXT, PREV, START, AVAIL, ITEM)

1. If START = -1 then
    - Print "Underflow" and Exit
2. PTR = START

3. Repeat while PTR != -1 and DATA[PTR] != ITEM
    - PTR = NEXT[PTR]
4. If PTR = -1 then
    - Print "Node not found" and Exit
5. If PREV[PTR] != -1 then
    - NEXT[PREV[PTR]] = NEXT[PTR]

    Else
    - START = NEXT[PTR] // Node was first
6. If NEXT[PTR] != -1 then
    - PREV[NEXT[PTR]] = PREV[PTR]
7. NEXT[PTR] = AVAIL // Release node
8. AVAIL = PTR
9. Exit

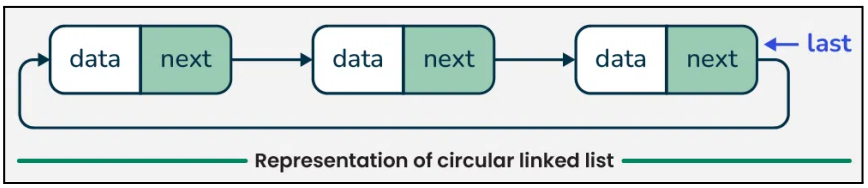
Circular Linked List

A Circular Linked List (CLL) is a special type of linked list in which the last node of the list points back to the first node, forming a circle. Unlike a singly or doubly linked list that ends with a NULL (or -1 in array representation), a circular linked list has no NULL pointer, since traversal can continue indefinitely in a circular manner.

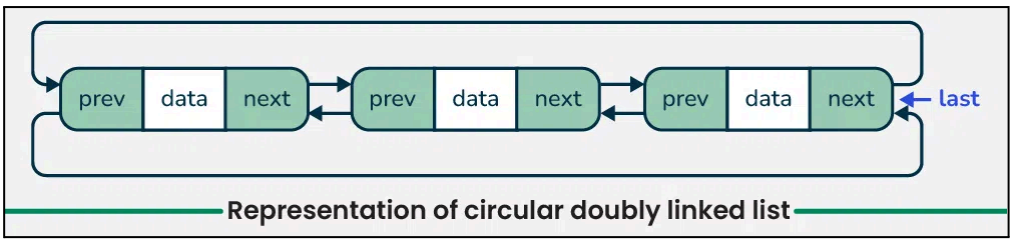
- **LAST** → points to the last node of the list.
- **LAST->NEXT** → always points to the first node (head).
- If **LAST = NULL**, the list is empty.

Characteristics of Circular Linked List

1. In a singly circular linked list, each node has two fields:
    - **DATA**: which stores the actual information.
    - **NEXT**: which stores the address of the next node.
    - The last node's **NEXT** pointer points back to the **first node**, creating a loop.



2. In a doubly circular linked list, each node contains three fields:
    - **DATA**: which stores the information.
    - **NEXT**: which points to the next node.
    - **PREV**: which points to the previous node.
    - The **NEXT** of the last node points to the first node, and the **PREV** of the first node points to the last node.



3. Because of the circular structure, traversal can start from any node and proceed in a circular fashion to visit all nodes.
4. There is usually no NULL in a circular linked list, so a special pointer such as HEAD (or TAIL) is maintained to know where the list starts.

Advantages of Circular Linked List

1. **Efficient Traversal:** You can start from any node and traverse the whole list in circular fashion without worrying about reaching the end.
2. **Useful for Applications:** It is widely used in applications like ***round robin scheduling*** (CPU scheduling), managing buffers (like circular queues), and in multiplayer games where players are arranged in a circle.
3. **Constant-Time Operations at Both Ends:** Insertion and deletion operations at the beginning and end can be done efficiently, especially when a ***tail pointer*** is maintained.
4. **Better Memory Utilization:** Unlike arrays, circular linked lists do not need a fixed size. They can grow or shrink dynamically as per requirement.

Disadvantages of Circular Linked List

1. **More Complex Implementation:** Handling pointers is more difficult compared to singly linked lists because of the circular connections.
2. **Infinite Loop Possibility:** Since there is no NULL termination, a careless traversal algorithm may go into an infinite loop if the stopping condition is not carefully checked.
3. **Extra Pointer Management:** More pointer updates are required during insertion and deletion.

Operations on Circular Linked List

Traversal in Circular Linked List

Algorithm: Traverse_CLL(LAST)

1. if LAST == NULL then
    - print "List is Empty"
    - return

   end if
2. Set PTR = LAST->NEXT // start from first node
3. repeat
    - Process PTR->DATA

    - Set PTR = PTR->NEXT
4. until PTR == LAST->NEXT
5. EXIT

Algorithm: InsertBegin(LAST, ITEM)

1. Create a new node NEW.
2. Set NEW->DATA = ITEM.
3. If LAST = NULL
    - Set LAST = NEW.
    - Set NEW->NEXT = NEW.
    - Exit.
4. Else
    - Set NEW->NEXT = LAST->NEXT.
    - Set LAST->NEXT = NEW.
5. Exit.

Algorithm: InsertEnd(LAST, ITEM)

1. Create a new node NEW.
2. Set NEW->DATA = ITEM.
3. If LAST = NULL
    - Set LAST = NEW.
    - Set NEW->NEXT = NEW. and Exit.
4. Else
    - Set NEW->NEXT = LAST->NEXT.
    - Set LAST->NEXT = NEW.
    - Set LAST = NEW.
5. Exit.

Algorithm: InsertAfter(LAST, ITEM, VALUE)

1. If LAST = NULL, display "List is Empty" and Exit.
2. Set PTR = LAST->NEXT.
3. Repeat
    - If PTR->DATA = VALUE then
        - Create a new node NEW.
        - Set NEW->DATA = ITEM.
        - Set NEW->NEXT = PTR->NEXT.
        - Set PTR->NEXT = NEW.
        - If PTR = LAST, then set LAST = NEW.
        - Exit.
    - Set PTR = PTR->NEXT.
      until PTR = LAST->NEXT.
4. Display "Value Not Found".
5. Exit.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Algorithm: DeleteBegin(LAST)

1. If LAST = NULL, display "Underflow" and Exit.
2. Set FIRST = LAST->NEXT.
3. If FIRST = LAST (only one node)
    - Free FIRST.
    - Set LAST = NULL.
    - Exit.
4. Else
    - Set LAST->NEXT = FIRST->NEXT.
    - Free FIRST.
5. Exit.

Algorithm: DeleteEnd(LAST)

1. If LAST = NULL, display "Underflow" and Exit.
2. If LAST->NEXT = LAST (only one node)
    - Free LAST.
    - Set LAST = NULL.
    - Exit.
3. Else
    - Set PTR = LAST->NEXT.
    - Repeat until PTR->NEXT = LAST
        - Set PTR = PTR->NEXT.
    - Set PTR->NEXT = LAST->NEXT.
    - Free LAST.
    - Set LAST = PTR.
4. Exit.

Algorithm: DeleteNode(LAST, VALUE)

1. If LAST = NULL, display "Underflow" and Exit.
2. Set CURR = LAST->NEXT and PREV = LAST.
3. Repeat
    - If CURR->DATA = VALUE then
        - If CURR = LAST and CURR->NEXT = LAST (only one node)
            - Free CURR.
            - Set LAST = NULL.
            - Exit.
        - Else
            - Set PREV->NEXT = CURR->NEXT.
            - If CURR = LAST, set LAST = PREV.
            - Free CURR.
            - Exit.
    - Set PREV = CURR.
    - Set CURR = CURR->NEXT.

until CURR = LAST->NEXT.

4. Display "Node Not Found".
5. Exit.

Array Representation of Circular Linked List (CLL)

In array representation, instead of pointers, we use two parallel arrays:

- **DATA[]** → stores the values.
- **NEXT[]** → stores the index of the next node.
- Here, **START** stores the index of the first node, and **NEXT[last]** will point back to START (making it circular).
- **AVAIL** is used to track free nodes, just like in array representation of normal linked lists.

Initial Circular Linked List

Example with 4 nodes (10 → 20 → 30 → 40, circular back to 10)

- START = 0 // List starts at index 0
- AVAIL = 4 // First free node is at index 4

Index	**0**	**1**	**2**	**3**	**4**
DATA	10	20	30	40	—
NEXT	1	2	3	0	—

Traversal: 10 → 20 → 30 → 40 → (back to 10)

Algorithm: Traverse_CLL(DATA[], NEXT[], START)

1. IF START = -1 THEN
    - PRINT "List is empty"
    - EXIT

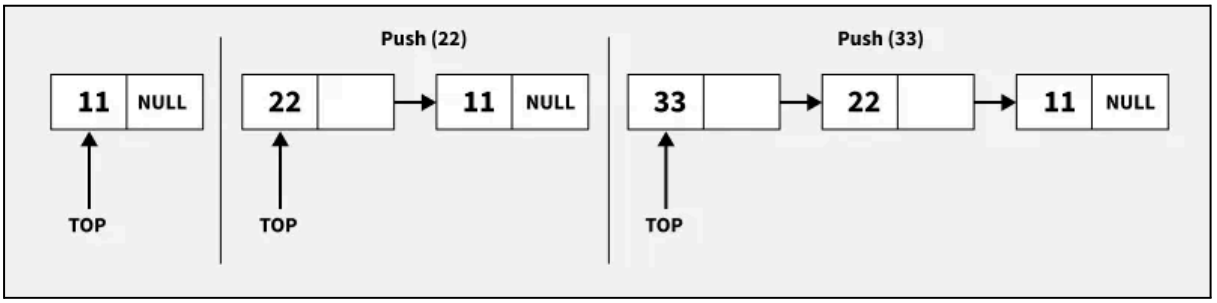
   ENDIF
2. SET PTR = START
3. REPEAT
    - PRINT DATA[PTR]
    - SET PTR = NEXT[PTR]

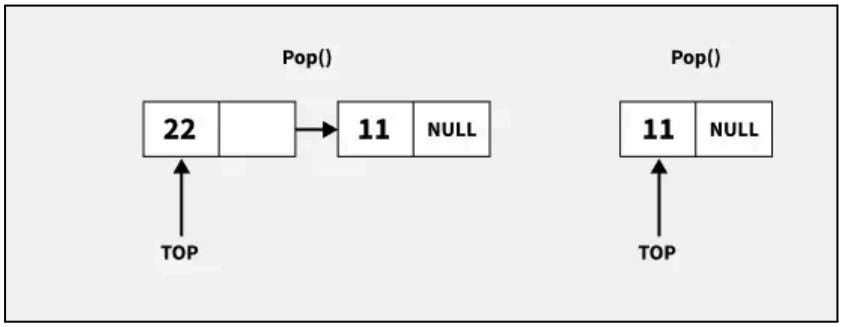
   UNTIL PTR = START
4. EXIT

Normal Linked List Representation of Stack

In a normal linked list representation of a stack, the stack is implemented using a singly or doubly linked list in a linear form. The **TOP** pointer always refers to the most recently inserted node, which represents the top of the stack. Insertion (**PUSH**) is usually done at the beginning of the list because it is efficient (constant time), and deletion (**POP**) is also performed from the beginning. Each node contains two parts: DATA (the value stored) and

NEXT (the address of the next node). Since the stack grows and shrinks dynamically, linked lists overcome the limitation of fixed size in array representation.





Code

```
// Check if stack is empty
int isEmpty(struct Node* head) {
    return head == NULL;
}

// Push an element onto stack
void push(struct Node** head, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = *head;
    *head = new_node;
}

// Pop the top element
void pop(struct Node** head) {
    if (isEmpty(*head)) return;
    struct Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}

// Return the top element
int peek(struct Node* head) {
    if (!isEmpty(head)) return head->data;
```

```
    return INT_MIN;
}
```

Applications

Used in recursion handling, expression evaluation, and memory management.

Advantages

- No overflow unless system memory is full.
- Dynamic size (stack grows/shrinks as required).
- Efficient PUSH and POP operations at the beginning.

Limitations

- Requires extra memory for storing links.
- Slightly more complex than array implementation.

Circular Linked List Representation of Stack

In a circular linked list representation, the nodes are arranged in such a way that the last node points back to the first node, forming a circle. The TOP pointer points to the most recent element. Since it is circular, we can make both ends of the stack accessible easily, though typically only TOP is used. The main advantage of using a circular linked list for stack implementation is that it allows traversal from TOP back to the beginning without requiring a separate HEAD pointer.

Applications

Useful when stacks are used in cyclic processes such as buffer management, scheduling, or when we want to reuse memory nodes efficiently.

Advantages

- Can traverse from top back to base without extra pointers.
- Efficient use of memory since nodes are circularly connected.
- Useful for implementing multiple stacks using the same memory pool.

Limitations

- Slightly more complex logic for insertion and deletion compared to linear list.
- Traversal requires care to avoid infinite looping.

Self Organizing Lists

A ***Self-Organizing List (SOL)*** is a type of linked list in which the order of elements is adjusted automatically based on how frequently or recently they are accessed. The main idea

is to keep frequently accessed elements near the beginning of the list so that searching becomes faster. Unlike a normal linked list where the order of elements is fixed unless explicitly changed, a self-organizing list dynamically rearranges itself after each access.

This technique is particularly useful in cases where some elements are searched much more often than others. By moving frequently used elements closer to the front, the average search time is reduced, even though occasional rearrangements are performed.

Common Strategies of Self-Organizing Lists

Move-to-Front (MTF)

- When an element is accessed, it is moved to the front (head) of the list.
- Frequently accessed elements stay near the beginning.
- Simple to implement, but may disturb order if recently used ≠ frequently used.

Algorithm: Search_MTF(HEAD, ITEM)

```
Set PREV ← NULL
Set PTR ← HEAD
While PTR ≠ NULL do
    If PTR->DATA = ITEM then
        If PREV ≠ NULL then
            PREV->NEXT ← PTR->NEXT
            PTR->NEXT ← HEAD
            HEAD ← PTR
        End If
        Return PTR   // found and moved to front
    End If
    PREV ← PTR
    PTR ← PTR->NEXT
End While
Return NULL   // ITEM not found
```

Transpose (Swap)

- When an element is accessed, it is swapped with its immediate predecessor.
- Over time, frequently accessed elements "bubble up" to the front.
- Less disruptive than MTF, but slower to adapt.

Algorithm: Search_Transpose(HEAD, ITEM)

```
Set PREV ← NULL
Set PTR ← HEAD

While PTR ≠ NULL do
    If PTR->DATA = ITEM then
```

```
        If PREV ≠ NULL then
            // Swap DATA values of PREV and PTR
            TEMP ← PREV->DATA
            PREV->DATA ← PTR->DATA
            PTR->DATA ← TEMP
            Return PREV   // item moved one step closer to front
        End If
        Return PTR   // item already at front
    End If
    PREV ← PTR
    PTR ← PTR->NEXT
End While

Return NULL   // ITEM not found
```

Count Method (Frequency Count)

- Each node keeps a counter of how many times it has been accessed.

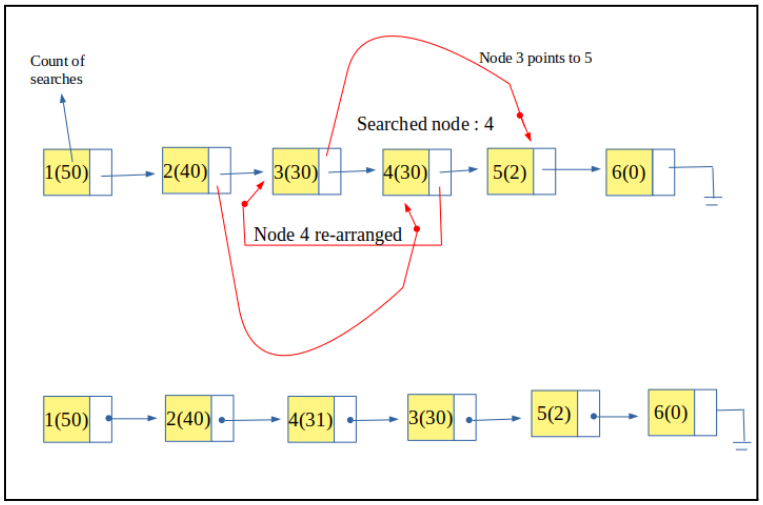
```
NODE {
    DATA
    NEXT
    COUNT   // used only in Count method
}
```

- After every access, the count is incremented, and the list is reorganized in decreasing order of access counts.
- Keeps most frequently used elements at the front, but requires extra storage and updates.

Algorithm: Search_Count(HEAD, ITEM)

```
Set PREV ← NULL
Set PTR ← HEAD

While PTR ≠ NULL do
    If PTR->DATA = ITEM then
        PTR->COUNT ← PTR->COUNT + 1

        // Reorganize list by count order
        If PREV ≠ NULL AND PREV->COUNT < PTR->COUNT then
            // Remove PTR from current position
            PREV->NEXT ← PTR->NEXT

            // Find new position for PTR
            Set Q ← HEAD
```

```
            Set QPREV ← NULL
            While Q ≠ NULL AND Q->COUNT ≥ PTR->COUNT do
                QPREV ← Q
                Q ← Q->NEXT
            End While

            // Insert PTR before Q
            If QPREV = NULL then
                PTR->NEXT ← HEAD
                HEAD ← PTR
            Else
                PTR->NEXT ← Q
                QPREV->NEXT ← PTR
            End If
        End If

        Return PTR   // item found and repositioned
    End If
    PREV ← PTR
    PTR ← PTR->NEXT
End While

Return NULL   // ITEM not found
```



Applications

- Used in caching systems, where frequently accessed items should be kept closer.
- Helpful in symbol tables in compilers or interpreters, where some identifiers are used repeatedly.
- Useful in text editors or command history, where recent or frequent actions are more likely to be repeated.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Skip Lists

A Skip List is a data structure that allows fast search, insertion, and deletion operations, similar to balanced binary search trees, but with a simpler and randomized approach. It was invented by ***William Pugh*** in ***1990*** as a probabilistic alternative to balanced trees. Skip lists use multiple "levels" of linked lists stacked on top of each other to speed up searching.

At the bottom level, we have a simple sorted linked list containing all elements. Above this, higher levels act as "***express lanes***" that skip over multiple nodes, allowing faster traversal. Each element is promoted to higher levels with a certain probability (usually 50%), which makes the structure balanced on average.

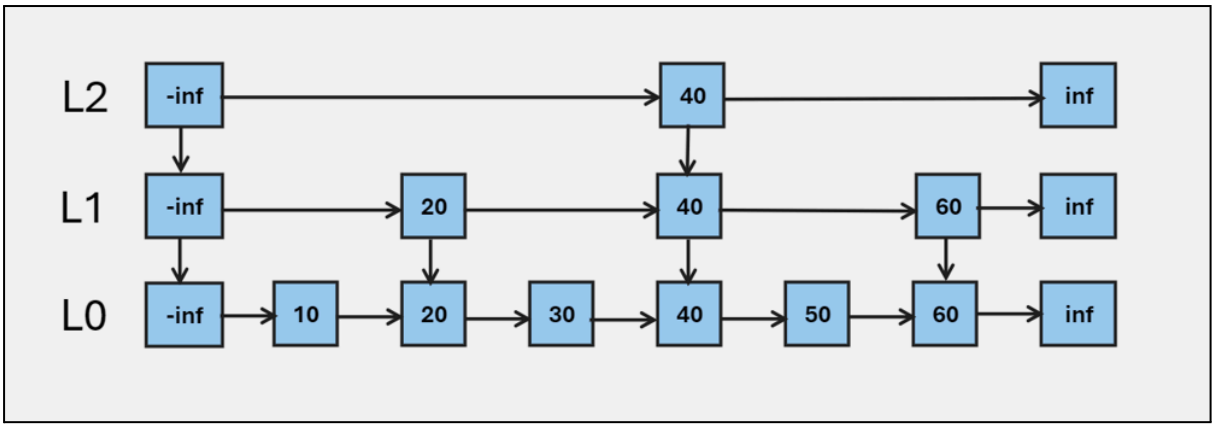
Think of a skip list like a highway with service lanes. The bottom lane is the local road (visiting every node), while upper lanes are express highways skipping many exits, helping you reach the destination faster.

Structure of a Skip List

Each node may contain multiple forward pointers: one for each level it belongs to.

- The bottom level contains all elements in sorted order.
- Higher levels contain fewer elements, but allow larger jumps.
- The top level contains the fewest nodes, sometimes just the head and tail.

Example



Why Skip Lists?

- In a sorted linked list, searching requires O(n) time since we must traverse nodes one by one.
- A balanced binary search tree (BST) allows search in O(log n) time, but its implementation is complex (rotations, balancing).
- A sorted array allows binary search in O(log n), but insertion/deletion is costly due to shifting.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

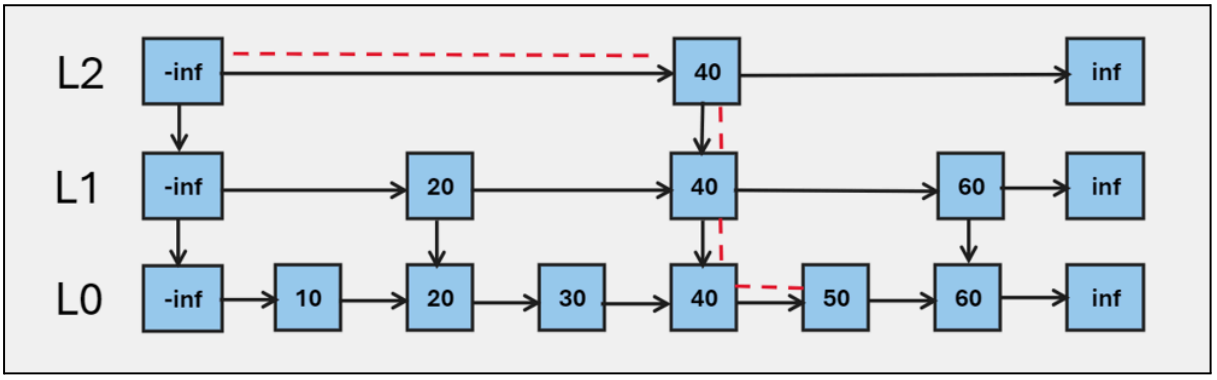
- Skip List solves this problem by augmenting a linked list with multiple layers of "express lanes", allowing nodes to be skipped during search.

Skip Lists combine the simplicity of linked lists with the efficiency of trees.

Operations in Skip Lists

Search

1. Start from the top level at the head node.
2. Move forward until the next node is greater than the target.
3. Drop down one level and continue.
4. Stop when the target is found or we reach NULL.
   Average time complexity: O(log n).



Algorithm: SkipListSearch(head, KEY)

1. Set CURRENT = head // Start from top-level head node
2. For each level from TOP → DOWN
   - While CURRENT->NEXT exists AND CURRENT->NEXT->DATA < KEY
     CURRENT = CURRENT->NEXT // Move forward
     End While
   - If CURRENT->NEXT exists AND CURRENT->NEXT->DATA == KEY
     Print "KEY Found"
     Return
   - Move CURRENT down one level
3. Print "KEY Not Found"
4. Exit

Insertion

1. Perform a search to find where the new element should be inserted.
2. Insert the element at the bottom level.
3. Randomly decide how many higher levels the element should appear in.
4. Insert the node at those higher levels too.

Algorithm: SkipListInsert(head, ITEM)

1. Create an array UPDATE[] to store path at each level
2. Set CURRENT = head

```
3. For each level from TOP → DOWN
    While CURRENT->NEXT exists AND CURRENT->NEXT->DATA < ITEM
        CURRENT = CURRENT->NEXT
    End While
    UPDATE[level] = CURRENT   // Remember the last node at this level
    Move CURRENT down one level
4. Create a new node NEW with DATA = ITEM
5. Randomly generate LEVEL for NEW (using coin flips)
6. For i = 1 to LEVEL
    Set NEW->NEXT[i] = UPDATE[i]->NEXT[i]
    Set UPDATE[i]->NEXT[i] = NEW
7. If LEVEL > MAXLEVEL
    Update MAXLEVEL = LEVEL
Exit
```

Deletion

- Search for the element across all levels.
- Remove references to it from every level in which it appears.

```
Algorithm: SkipListDelete(head, KEY)
1. Create an array UPDATE[] to store path at each level
2. Set CURRENT = head
3. For each level from TOP → DOWN
    While CURRENT->NEXT exists AND CURRENT->NEXT->DATA < KEY
        CURRENT = CURRENT->NEXT
    End While
    UPDATE[level] = CURRENT
    Move CURRENT down one level
4. CURRENT = CURRENT->NEXT
5. If CURRENT == NULL OR CURRENT->DATA != KEY
    Print "KEY Not Found"
    Return
6. For i = 1 to MAXLEVEL
    If UPDATE[i]->NEXT[i] != CURRENT
        Break   // Node not present in this level
    UPDATE[i]->NEXT[i] = CURRENT->NEXT[i]
7. Delete CURRENT
8. While MAXLEVEL > 1 AND head->NEXT[MAXLEVEL] == NULL
    MAXLEVEL = MAXLEVEL - 1
Exit
```

Advantages of Skip Lists

- Skip lists are simple, solid, and trustworthy compared to complex balanced trees.
- Insertion of new elements is extremely quick because levels are decided randomly.

- Easier to implement than hash tables or balanced binary search trees.
- Average time complexity for search, insert, and delete is O(log n), similar to balanced trees.
- As the number of nodes increases, the probability of worst-case performance decreases.
- Finding a node is straightforward since traversal is just "express lane + normal lane."

Limitations

- Skip lists require more memory than simple linked lists, since multiple forward pointers are stored.
- Reverse traversal is not supported, unlike doubly linked lists.
- In the worst case (unlucky randomization), search may degrade to O(n).
- Slightly less cache-friendly compared to arrays, because nodes are scattered in memory.